

Práctica de Examen

Curso

Programación Intermedia en Python

Primera parte

Seleccione la opción correcta en cada caso. Cada pregunta tiene un único valor correcto.

1. ¿Cuál de los siguientes es un tipo de dato válido en Python?
 - a) integer
 - b) number
 - c) int**
 - d) character
2. ¿Qué función permite recibir información ingresada por el usuario?
 - a) print()
 - b) input()**
 - c) read()
 - d) scan()
3. ¿Qué operador permite obtener el residuo de una división?
 - a) /
 - b) //
 - c) %**
 - d) **
4. ¿Cuál estructura se utiliza para evaluar una condición?
 - a) for
 - b) if**
 - c) while
 - d) input
5. ¿Qué instrucción permite finalizar un ciclo antes de que termine normalmente?
 - a) stop
 - b) exit
 - c) break**
 - d) continue
6. ¿Qué función permite abrir un archivo en Python?
 - a) file()
 - b) open()**
 - c) read()
 - d) load()
7. ¿Cuál es el propósito principal de una clase?
 - a) Crear únicamente variables globales
 - b) Agrupar datos y comportamientos**
 - c) Eliminar archivos
 - d) Ejecutar ciclos automáticamente

8. ¿Qué método se utiliza normalmente para inicializar una clase?
 - a) `__start__()`
 - b) `__create__()`
 - c) `__init__()`
 - d) `init()`
9. ¿Qué permite la herencia?
 - a) Eliminar una clase padre
 - b) Crear una clase a partir de otra existente
 - c) Convertir una lista en un diccionario
 - d) Crear únicamente funciones
10. En una relación de herencia, ¿cómo se denomina la clase que hereda?
 - a) Clase principal
 - b) Clase padre
 - c) Clase hija
 - d) Clase base obligatoriamente
11. ¿Qué función permite acceder a métodos o al constructor de la clase padre?
 - a) `parent()`
 - b) `super()`
 - c) `base()`
 - d) `father()`
12. ¿Qué sucede cuando una clase hija redefine un método heredado?
 - a) Se elimina la clase padre
 - b) Se realiza una sobreescritura del método
 - c) Se convierte en una función global
 - d) Se elimina el objeto
13. ¿Qué es una Serie en Pandas?
 - a) Una tabla bidimensional
 - b) Una columna unidimensional
 - c) Un archivo CSV
 - d) Una matriz tridimensional
14. ¿Qué estructura representa una tabla con filas y columnas en Pandas?
 - a) Series
 - b) DataFrame
 - c) Array
 - d) Table
15. ¿Qué método permite obtener información general sobre un DataFrame?
 - a) `describe()`
 - b) `info()`
 - c) `data()`
 - d) `information()`
16. ¿Qué método permite obtener el valor máximo de una columna?

- a) maximum()
 - b) top()
 - c) max()
 - d) high()
17. ¿Qué método permite eliminar registros duplicados?
- a) remove_duplicates()
 - b) drop_duplicates()
 - c) delete_duplicates()
 - d) duplicates_remove()
18. ¿Qué método permite calcular la correlación entre las columnas de un DataFrame?
- a) relation()
 - b) correlation()
 - c) corr()
 - d) compare()
19. ¿Qué método puede utilizarse para calcular la media de una columna?
- a) average()
 - b) mean()
 - c) middle()
 - d) avg()
20. ¿Cuál de los siguientes corresponde a un tipo de gráfico disponible mediante plot()?
- a) scatter
 - b) matrix
 - c) database
 - d) table

Segunda parte

1. Explique con sus propias palabras:
- a. ¿Qué es una clase?
Es una forma de modelar una entidad, sus atributos y acciones en programación.
 - b. ¿Qué es un objeto?
Es una instancia de una clase que representa una entidad concreta y que contiene valores específicos para sus atributos.
 - c. ¿Cuál es la función del método `__init__()`?
Es el constructor de una entidad, el cual permite crear un objeto.
 - d. Mencione un ejemplo sencillo de una clase que podría utilizarse en un sistema informático.
Una clase usuario, le cual contenga el rol de usuario, el correo, nombre y que tenga las acciones que pueda hacer el usuario.

2. Considere las clases:

```
class Persona:
    def __init__(self, nombre):
        self.nombre = nombre

class Estudiante(Persona):
    def __init__(self, nombre,
carrera):
        super().__init__(nombre)
        self.carrera = carrera
```

- ¿Cuál es la clase padre?
Persona
 - ¿Cuál es la clase hija?
Estudiante
 - ¿Qué función cumple `super()`?
Llamar el constructor de la Persona.
 - ¿Qué atributo recibe la clase Estudiante de la clase Persona?
El nombre.
3. Explique tres problemas que pueden encontrarse al analizar un dataset y cómo podrían solucionarse utilizando Pandas.
- Datos vacíos, datos erróneos o formatos incorrectos. Para eso se puede utilizar funciones de pandas como `dropna`, para eliminar los datos vacíos y aplicar condiciones sobre cada filar para verificar que cumplan los formatos como umbrales.
4. Explique qué significa analizar la correlación entre dos variables.
- Significa estudiar si existe una relación entre dos variables y qué tan fuerte es esa relación, es decir, determinar si cuando una variable cambia, la otra tiende a cambiar también.

Tercera parte

Sistema de estudiantes y análisis de datos

Una universidad desea desarrollar un pequeño programa para registrar estudiantes y posteriormente analizar sus datos.

Parte A – Programación orientada a objetos

Cree una clase llamada Estudiante que contenga:

- nombre
- edad
- estatura
- horas_estudio
- calificacion

La clase debe tener:

- Un constructor `__init__()`.
- Un método `mostrar_datos()` que muestre la información del estudiante.

Luego:

- Cree al menos 5 objetos de tipo Estudiante.
- Guarde los objetos en una lista.
- Utilice un ciclo for para mostrar la información de todos los estudiantes.

Parte B – Pandas

A partir de los estudiantes registrados, cree un DataFrame con las columnas:

- Nombre
- Edad
- Estatura
- HorasEstudio
- Calificacion

Realice las siguientes operaciones:

- Mostrar el DataFrame.
- Mostrar la información general del DataFrame.
- Obtener el promedio de:
 - Edad
 - Estatura
 - HorasEstudio
 - Calificacion

- Obtener el estudiante con la mayor calificación.
- Obtener el estudiante con la menor calificación.
- Calcular la matriz de correlación utilizando corr().
- Analizar la correlación entre:
 - HorasEstudio y Calificacion
 - Estatura y Edad
 - Crear un gráfico de dispersión entre HorasEstudio y Calificacion.

```
import pandas as pd
import matplotlib.pyplot as plt

# =====
# Clase Estudiante
# Representa a un estudiante y almacena sus datos.
# =====
class Estudiante:
    def __init__(
        self,
        nombre: str,
        edad: int,
        estatura: float,
        horas_estudio: int,
        calificacion: float
    ):
        self.nombre = nombre
        self.edad = edad
        self.estatura = estatura
        self.horas_estudio = horas_estudio
        self.calificacion = calificacion

    def mostrar_datos(self):
        """Muestra en pantalla los datos del estudiante."""
        print(f"Nombre: {self.nombre}")
        print(f"Edad: {self.edad}")
        print(f"Estatura: {self.estatura}")
        print(f"Horas de estudio: {self.horas_estudio}")
        print(f"Calificación: {self.calificacion}")

# =====
# Crear una lista para almacenar los estudiantes
# =====
estudiantes = []

# =====
# Solicitar los datos de 5 estudiantes
# =====
for i in range(5):
    print(f"\n--- Estudiante {i + 1} ---")

    estudiante = Estudiante(
        input("Nombre: "),
        int(input("Edad: ")),
        float(input("Estatura: ")),
        int(input("Horas de estudio: ")),
        float(input("Calificación: "))
    )

    estudiantes.append(estudiante)

# =====
# Convertir los objetos Estudiante en un DataFrame
```

```
# =====

# __dict__ permite obtener los atributos del objeto
# como un diccionario.
datos = []

for estudiante in estudiantes:
    datos.append(estudiante.__dict__)

df = pd.DataFrame(datos)

# =====
# Mostrar los datos registrados
# =====
print("\n===== DATOS DE LOS ESTUDIANTES =====")
print(df)

# =====
# Calcular el promedio de las variables numéricas
# =====
print("\n===== PROMEDIOS =====")
print(df.mean(numeric_only=True))

# =====
# Encontrar el estudiante con la calificación más alta
# =====
indice = df["calificacion"].idxmax()

print("\n===== MEJOR CALIFICACIÓN =====")
print(f"Calificación máxima: {df['calificacion'].max()}")
print("Datos del estudiante:")
print(df.loc[indice])

# =====
# Encontrar el estudiante con la calificación más baja
# =====
indice = df["calificacion"].idxmin()

print("\n===== MENOR CALIFICACIÓN =====")
print(f"Calificación mínima: {df['calificacion'].min()}")
print("Datos del estudiante:")
print(df.loc[indice])

# =====
# Analizar la correlación entre las variables numéricas
# =====
print("\n===== MATRIZ DE CORRELACIÓN =====")
print(df.corr(numeric_only=True))

# =====
# Crear un gráfico de dispersión
# Relaciona las horas de estudio con la calificación.
# =====
df.plot(
    kind="scatter",
    x="horas_estudio",
    y="calificacion"
)
plt.show()
```